School of Electronic Engineering
and Computer Science

Final Report

Programme of study:
Computer Science

Project Title:
Control Flow Graph Analysis using Graph Edit Distance for Cybersecurity Applications

Final Year
Undergraduate Project 2025/26



Supervisor:
Michael Shekelyan

Student Name:
Artur Istratov

Date: 06/05/2026

Abstract

Traditionally, the detection of malware relies on the use of signatures, fingerprints of known malware binaries used to detect similarities between them and new programs. To bypass this method, attackers make modifications in the program to change surface characteristics without altering program functionality by means of renaming variable names, modifying control flow, adding no-op instructions, etc. The problem with the traditional signature-based detection technique is that it breaks down under even the most basic transformation, because it is based on syntax. A Control Flow Graph (CFG) represents a branching structure of a program and is more robust to transformations; therefore, CFG-based techniques can be used for malware detection. This project studies the applicability of CFG-based similarity comparison using the Graph Edit Distance (GED) to classify variants of a Python program (i.e., malware with topological transformations) from a baseline (clean) program, as well as investigate if the choice of the edit cost model influences the classification results.

The research question of the work is the following: Is it possible to reliably classify variants of a Python program using the Graph Edit Distance with two different edit cost models? CFG is constructed at the block level based on the Python bytecode of a program; the graph is normalised by assigning roles, collapsing linear chains, and canonical labelling. Then exact GED is calculated using two edit cost models: a uniform model and a non-uniform cost model where the weight of branch nodes and jump edges is increased. Two exact GED algorithms (Dijkstra GED and A* GED using a Hungarian lower-bound heuristic) along with an adaptive racing strategy are evaluated. As the result, out of 12 samples (9 obfuscated variants of one baseline function and two structurally unrelated programs), the algorithm classifies 9 samples correctly (82% accuracy, 89% precision, and 89% recall). In topology-preserving obfuscations (variable renaming, dead code, no-op instructions, expression splitting), GED equals zero after normalising, and thus the algorithm is insensitive to those transformations. In topology-changing transformations (loop unrolling, opaque predicate, bogus exception handling, control flow flattening), non-zero GED values are produced and classified below the variant threshold. There is only one false negative (the sample uses several hard obfuscation methods), and one false positive sample (structurally unrelated and similar in size). In addition, the non-uniform model significantly amplifies GED values by 54% but leaves the classification results unchanged.

Contents

Chapter 1: Introduction 4
1.1 Background 4
1.2 Problem Statement 5
1.3 Aim 5
1.4 Objectives 5
1.5 Research Questions 6
1.6 Report Structure 6
Chapter 2: Literature Review 7
2.1 Control Flow Graphs in cybersecurity 7
2.2 Graph Edit Distance algorithms 9
2.3 Integrating CFGs and GED for malware analysis 11
2.4 Summary and research gap 12
Chapter 3: Evaluation methodology 14
3.1 System architecture 14
3.2 CFG extraction 16
3.3 CFG normalisation 16
3.4 Cost models 18
3.5 GED algorithms 19
3.6 Size-normalised GED and variant classification 21
3.7 Evaluation pipeline 21
Chapter 4: Results 23
4.1 CFG structural properties of the dataset 23
4.2 GED scores and obfuscation detection 24
4.3 Non-uniform cost model sensitivity 27
4.4 Algorithm efficiency: A* vs Dijkstra 28
4.5 Size-normalised GED and variant classification 30
Chapter 5: Evaluation and discussion 31
5.1 Research question answers 31
5.2 Limitations 33
5.3 Relationship to prior work 34
5.4 Threats to validity 34
Chapter 6: Conclusion 36
Chapter 7: References 38
Appendix A – Worked Example: GED computation for s01 vs s06 40
Appendix B – Gen AI Declaration 41

Chapter 1: **Introduction**

Malware developers often modify code to evade detection without significantly altering the program behaviour. For instance, renaming variables, adding unused code, filling with no-operation instructions, opaque predicates, exceptions, and control-flow flattening can create two related programs that differ in textual and byte-level representations.
This is an issue for signature-based, byte-based, or even textual approaches to detecting malicious software. Such methods may react to surface-level code differences without any real effect on control-flow. Thus, a more abstract representation is required.
The Control Flow Graph (CFG) models program behaviour by describing all possible execution paths through the application. Nodes and edges in the CFG stand for basic blocks and their control-flow connections, respectively. CFG provides a solid basis for program comparison due to preserving the branching, looping, merging, and halting semantics while omitting most of the syntax.
Graph Edit Distance (GED) quantifies the dissimilarity between two graphs as the minimum cost of graph transformation via node/edge addition, deletion, and substitution. For this project, GED is used to evaluate the control flow modifications of normalized CFGs caused by various obfuscation strategies.
The project explores GED-based program comparisons by focusing on edit cost design. While existing literature utilizes only uniform costs or neglects analysis of the impact of the choice of cost model on CFG similarity measurements, this study considers both uniform and non-uniform models. Specifically, the latter prioritises control-flow elements such as branch nodes and jump edges over other nodes and edges.
The proposed system constructs CFGs at the block level from Python bytecode, normalizes the graphs, and calculates the exact GED by applying the Dijkstra- and A* - based search algorithms. The evaluation will involve twelve Python scripts with controlled transformations, including simple obfuscations, complicated structural transformations, and unrelated programs serving as negative controls. The project goal is not to develop an operational malware detection mechanism but to demonstrate the CFG + GED approach.

1.1 Background

Malware, along with other kinds of damaging software, is often altered using techniques such as control flow flattening, compiler optimization, or obfuscation, which change the form while keeping the functionality of the program intact. Current tools, which rely on binary signatures (specific byte sequences in the compiled executable) or textual patterns, fail to detect these kinds of alterations. A structure-based method is necessary, which can measure behavioural changes between different programs irrespective of their form. This project aims

to create a system that can build control flow graphs from Python programs, measure structural changes using graph edit distance, which can improve the detection and classification of malware.

1.2 Problem Statement

Malware developers often change their software using techniques such as recompilation, instruction reordering, or control flow obfuscation, which makes it hard to detect malware. These changes often trick traditional detection tools that depend on binary or textual signatures. Despite their different appearances, the control flow graph of malware often looks similar to that of legitimate software.
This project attempts to solve the problem of detecting structural similarities between different kinds of software. It does so by using control flow graphs to represent the structure of different kinds of software, which can be quantitatively compared using graph edit distance. This will allow the system to detect different versions of malware that, despite their different appearances, retain similar control flow structures – because the author needs the variant to perform the same task (exfiltrate data, encrypt files, open a backdoor), which requires specific loops, branches, and error-handling paths that are hard to change without breaking the malware's functionality.

1.3 Aim

The aim of this project is to create and evaluate a control flow graph (CFG)-based system to measure the similarity of structure in Python code, using exact graph edit distance to investigate the impact of control flow normalization and edit cost definition on malware variant-style code comparison.

1.4 Objectives

1. Review existing literature on graph edit distance techniques and control flow graphs in the context of malware detection, and examine if prior work has studied the effect of edit cost model choice on GED-based similarity measurement.
2. Implement an analytical system using Python that can build control flow graphs from program code.
3. Apply graph edit distance algorithms to measure structural differences between control flow graphs.
4. Evaluate the accuracy of the system using different versions of program code.
5. Assess the applicability of CFG + GED analysis for malware variant detection and classification, measured by detection accuracy across obfuscation levels and the system's ability to distinguish genuine variants from structurally different programs.

1.5 Research Questions

1. To what extent do block-level CFGs preserve structural similarity under superficial code obfuscation?
2. Can exact GED classify Python function variants under different levels of control-flow obfuscation?
3. Which obfuscation techniques are removed by CFG normalisation, and which continue to produce measurable structural change?
4. How does a non-uniform edit cost model affect GED scores compared with a uniform cost model?
5. Does the edit cost model selection influence the effectiveness of pruning and execution speed of A*-GED over Dijkstra-GED?

1.6 Report Structure

The report will start by giving a background and literature survey on Control Flow Graphs and GED techniques in cybersecurity. This will be followed by a detailed discussion on the methodology adopted for conducting this research. At the end of the report, there will be a discussion on the findings, limitations, ethical concerns, and future research directions.

Chapter 2: **Literature Review**

This chapter will include a literature survey on Control Flow Graphs in cybersecurity, GED techniques, and their application to malware detection by combining CFG-based representation and GED-based similarity measurement. This literature survey will also identify a research gap in existing literature, which will be filled by this research project.

2.1 Control Flow Graphs in cybersecurity

2.1.1 Foundations of control flow analysis

In a paper from 1970, Allen introduced the concept of the Control Flow Graph as a directed graph whose nodes are the basic blocks of a sequence of instructions, and edges are the possible flows between basic blocks. He also proposed the definition of a leader instruction and the well-known basic blocks extraction algorithm that are still in use today. This paper laid the foundation for the use of control flow analysis in compiler optimisation, which includes data flow analysis, loop detection, and register allocation.
The relevance of Allen's paper to this project lies in two aspects. Firstly, the basic blocks extraction algorithm described in the paper is the starting point of the algorithm implemented in cfg_blocks.py. Secondly, Allen describes linear chain collapsing as a standard reduction step. This reduction is part of the normalisation pipeline in this project as well.
The usefulness of the CFG for program analysis is due to its abstract nature. Two programs with the same branching structure will yield the same CFG regardless of the way they are written, even though this may not always hold true. Programs that implement the same functionality recursively and iteratively may end up having different CFGs. Moreover, the CFG does not account for variable names, constant values, and the order of instructions inside basic blocks, which is what makes it suitable for malware detection. Attackers can modify the surface syntax to bypass signature-based detection, but it will be hard to modify the CFG without changing the program's functionality.

2.1.2 Malware detection based on CFG similarity

An early application of CFG similarity to malware variant detection is by Kruegel et al. (2005). They demonstrated that the branching structure persists in malware binaries after common obfuscation steps, including instruction reordering, register renaming, and junk insertion. CFG-based analysis proved significantly superior to signature-based detection on obfuscated malware variants. Bruschi, Martignoni, and Monga (2006) proposed a related approach focused specifically on self-mutating malware, using CFG matching to detect variants whose binary

representation changes between executions while their underlying control flow remains similar.
The empirical claim that CFG-based methods perform better than signature-based methods due to their reliance on control flow structure was the key contribution that led to this field's emergence.
Cesare and Xiang (2011) took it further by designing a CFG similarity search system for malware variant detection. They decomposed CFGs into features of the form of subgraphs and performed nearest neighbour search on a large malware corpus. They found evidence that CFG-based methods are indeed superior to signature-based methods on obfuscated malware variants. From their findings, one important observation is relevant to this project: obfuscation leads to increased complexity by adding new nodes and edges to the CFG without altering the overall control flow structure. This is the observation behind the insertion penalty in the non-uniform cost model.
One common issue in Kruegel et al. (2005) and Cesare and Xiang (2011) is that they applied CFG-based detection on binary executables compiled from C/C++. In this project, the CFGs are extracted from binary code written in Python at the level of bytecode. The definition of a basic block in bytecode differs from the assembly language version used by Kruegel et al., and the Python compiler performs dead-code elimination and constant folding before compiling to bytecode. As such, the normalisation process must take care of Python-related issues that were not present in the aforementioned studies.

2.1.3 Comparison of binary files and semantic differencing

BinHunt (Gao et al., 2010) extended CFG-based comparison of binaries from structural similarity to semantic differencing. Specifically, BinHunt verifies the semantic equivalence of basic blocks through symbolic execution, a step that yields more accurate results but also increases the computational cost greatly. What matters here is the trade-off between precision and cost: while semantic differencing based on symbolic execution is infeasible in this context, structural GED is feasible for comparing functions implemented in Python.
Another study that uses CFGs for detecting malware obfuscation is by Ugarte-Pedrero et al. (2014). They demonstrated that packed binaries can be identified through structural and heuristic features of executables and showed that anomaly detection over features such as block count, branching factor, and entropy patterns can flag packed files without unpacking them.
Their finding provides a justification for the role-based approach in the non-uniform cost model: the block type distribution contains information on the extent of the obfuscation.

2.1.4 Reconstruction of CFGs from obfuscated binaries

Yakdan et al. (2015) address CFG reconstruction from heavily obfuscated binaries, particularly those subjected to control flow flattening. In control flow flattening, a dispatcher substitutes for the normal control flow mechanism, yielding CFGs that are difficult to interpret with ordinary disassemblers. They

proposed a pattern-independent algorithm for restructuring control flow into readable code without knowledge of the used obfuscation technique. In this project, similar effect is observed: the s07_hard_flatten sample generates a difficult-to-interpret CFG, and the GED value of 34.0 against the clean baseline corresponds to the structural difference caused by the obfuscation.

2.1.5 Deep learning techniques for CFG-based detection

Recent methods employ Graph Neural Networks (GNNs) for training classifiers using labelled datasets of malware samples. For instance, Yan et al. (2019) employed a deep graph convolutional network for learning malware control flow graphs (CFGs), where each basic block was represented as a node, whose feature vector was obtained from the block instructions, learning the classification model in an end-to-end manner.
While GNN-based and GED-based approaches can complement each other in practical applications, they cannot substitute each other: the former requires labelled training sets and produces uninterpretable distance measures, whereas the latter requires neither training nor labelling and provides meaningful distances that can be decomposed into contributions of individual nodes and edges. This makes the latter preferable in the context of this project, which requires interpretability. On the other hand, the former would be preferred for building a production classification system with enough labelled examples available.

2.2 Graph Edit Distance algorithms

2.2.1 Theoretical foundations

Bunke (2000) laid the foundation for GED and proved several properties: it is a metric (non-negative, identity of indiscernible, symmetric, satisfies triangle inequality) under symmetric cost functions, it generalises graph isomorphism under binary cost function values, and it generalises string edit distance under path graphs. Bunke (2000) also proved that GED is NP-hard.
The implication of NP-hardness is that exact GED algorithms have exponential run-time complexity in the size of the input graphs, so there must be a trade-off between exact and tractable. This is the reason behind choosing block-level over instruction-level CFGs (three to five nodes vs up to 27) and the reason why the most time-consuming match in the evaluation dataset (s01 vs s10, four blocks vs 27) needs more than eight seconds with A* search pruning.

2.2.2 Exact GED algorithms

The classical approach to compute GED is the best-first search over the space of partial node mappings, which was originally devised for error-correcting graph matching. States consist of partial bijective node maps with null entries representing insertions or deletions. A state is expanded by matching each unmatched source node to every possible target node or null. The search stops

once all source nodes have been matched, and the cost (edit cost for the mapping and edge edits) of the mapping produced by the final state is a candidate for the final GED value.
But what makes a state better or worse?
One way to evaluate the progress is Dijkstra's algorithm: it expands states by increasing committed cost, and the result is guaranteed optimal but exhaustive. A* search combines the cost with an admissible lower-bound estimate of the remaining cost, enabling the search to prune away states whose best-case completion will cost too much. That is, an admissible heuristic prevents the search from visiting states that are no better than the current best state, thus preserving optimality.
There has been considerable effort put into improving the heuristics for A* GED search. The consistent lesson learnt in this regard is that a good heuristic can significantly reduce the number of states searched, especially in graphs with heterogeneous labels allowing tighter lower-bound estimates.

2.2.3 Approximate GED via bipartite matching

Given the computational intractability of exact GED algorithms on large graphs, many studies have developed approximate algorithms that are much faster at the cost of suboptimality. The approximate GED algorithms proposed by Neuhaus, Riesen, and Bunke (2006) are based on the observation that the assignment problem can be approximated via bipartite matching: a cost matrix is created in which each entry represents the cost of matching a particular source node to a particular target node or null, and the optimal assignment of source nodes is found in $O(n^3)$ via the Hungarian algorithm. This yields a good upper bound on GED, particularly for highly similar or dissimilar graphs.
Fankhauser, Riesen, and Bunke (2011) generalised this idea to solve bipartite GED in large graphs (hundreds of nodes). Riesen and Bunke (2009) used bipartite GED as the graph-distance measure for approximate nearest-neighbour searching in embedding graphs into vector spaces. All three approaches are directly applicable to this project, with the function ged_approximate() in ged.py providing the approximate computation of GED via bipartite matching in $O(n^3)$. Another open question is the relation between exact and approximate GED algorithms and their proximity to optimality.

2.2.4 Hungarian algorithm as a GED heuristic

The Hungarian algorithm (Kuhn, 1955) is a linear assignment algorithm with a worst-case time complexity of $O(n^3)$. It can find the global minimum cost assignment between any two sets of nodes in a graph. In A* search for GED, it is used to compute the admissible heuristic h(s), that is, the lowest cost required to match the unmatched nodes in state s. Riesen and Bunke (2009) proved that the minimum cost of completing the optimal bipartite assignment of unmatched nodes is a lower bound for the remaining node cost, given that edge cost is always non-negative.

This project's implementation of h(s) improves the standard heuristic by considering the difference in out-degrees between the matched temporary nodes and adding the additional minimum cost of handling excess out-edges. Therefore, the implementation of h(s) returns a tighter lower bound compared to the standard node-only heuristic.

2.2.5 GED surveys and taxonomy

Gao, Xiao, Tao, and Li (2010) provide a survey of GED algorithms, classifying them into two broad types: exact and approximate algorithms. The classification provides context for understanding the position of the algorithms used in this project: the exact A* algorithm fits into the first group, while the bipartite matching approximate algorithm fits into the second group.

Learning-based graph distance calculation appears in more recent CFG analysis literature (Yan et al., 2019, Section 2.1.5), postdating Gao et al.'s survey. As Gao et al. observe, one issue with existing GED algorithms, reiterated in this survey, is a lack of research into GED cost function design and its impact on GED-based distance.

2.3 Integrating CFGs and GED for malware analysis

2.3.1 Structural similarity for malware classification

Narayanan et al. (2016) introduced the idea of using graph kernels, particularly the Contextual Weisfeiler-Lehman kernel, to distinguish between malware using program graph models. It was found that the intra-family graphs cluster more closely than inter-family graphs, giving an empirical indication that structure and behaviour are similar.

The problem that is being tackled by this project is that current research on graph kernels and edit distance within this domain, such as Narayanan et al. (2016), assumes static weighting of structure components and does not perform a sensitivity analysis on the weights used.

2.3.2 Effect of obfuscation on CFG-based similarity

Cesare & Xiang (2011) evaluated their algorithm for detecting CFG similarities under instruction substitution, register renaming, junk insertion, and partial control flow restructuring and noted gradual decrease in similarity levels with increasing obfuscation. Similarly, in this project, GED equalled 0 when the level of difficulty of obfuscation was low and was between 20 and 60 for high levels of obfuscation. Obfuscations involving control flow flattening for purposes of decompilation were examined by Yakdan et al. (2016) who determined that control flow flattening destroyed the hierarchy of control flow and replaced it with a sequence of conditional branches, yielding a CFG with very different structure. The high GED of 34.0 generated for s07 indicates agreement with their observation that

flattening represents a fundamental obfuscation technique. Fig. 3.2 depicts side-by-side canonical CFGs for s01_clean and s07_hard_flatten.
Unlike flattening, the effect of opaque predicates on CFG-based similarity is minimal since there is no alteration of the structure of the paths – only unnecessary nodes representing conditionals are added. The GED of 20.0 achieved for s08 is indicative of those nodes, despite the absence of any change in the actual path taken by the program, as detection of the fact that they represent constant branches requires either dynamic or symbolic analysis.

2.3.3 Scalability of CFG + GED systems

In practice, real malware binaries have many functions with huge numbers of basic blocks so that comparison between CFGs would be computationally expensive. However, such issue was tackled by Gao, Reiter, and Song (2008) in their BinHunt algorithm using function signatures. Alternatively, Riesen and Bunke (2009) proposed bipartite-matching approximate GED to speed up comparison of large graphs.The present project is different in that the purpose of analysis is different from those in the above works, enabling to use exact GED in comparison of the CFGs – by performing static analysis of Python bytecode whose CFGs are small and analysing function-level CFGs of relatively small and simple programs. Hard obfuscation examples (s07-s10) present a challenge to scalability and adaptive racing, running both exact and approximate method simultaneously and accepting the output of whichever completes first, solves the issue.

2.3.4 Cost model design in program similarity

Very little literature exists concerning design of cost model in program similarity algorithms. According to Gao et al. (2010), uniform unit cost is the default model where insertion or deletion of every node/edge costs one ”point”.
Although uniform unit cost model allows bypassing of domain-specific design and simplifies theoretical analysis, it generates metric that treats all types of edit operations uniformly, thus weighting changes in the branching of CFG (deletion of a BRANCH) the same as changes in linear structure (deletion of a LINEAR).
Non-uniform cost model has been used on domain-specific grounds but without systematic analyses. For example, although Cesare & Xiang (2011) used non-uniform cost in the evaluation of the substructure features, they did not explicitly analyse them as a cost model. In addition, Narayanan et al. (2016) designed graph kernels that assign different importance to different substructure components but did not analyse the sensitivity of the result to those weights.Thus, lack of systematic analyses of cost model choice motivates this project.

2.4 Summary and research gap

Literature survey on CFG similarity provides theoretical and empirical background on the use of CFGs together with GED for the purpose of comparing malware. The theoretical foundation of CFGs was provided by Allen (1970) while

their robustness to obfuscation in binary malware was demonstrated by Kruegel et al. (2005) and Cesare & Xiang (2011). The extension of CFG analysis to semantic differencing and packing detection was proposed by Gao et al. (2008) and Ugarte-Pedrero et al. (2014). CFG reconstruction and classification of malware using graph neural networks was performed by Yakdan et al. (2015) and Yan et al. (2019), respectively.

Theoretical foundations of GED were described by Bunke (2000), Neuhaus et al. (2006), Riesen & Bunke (2009), and Fankhauser et al. (2011). Gao, Xiao, Tao, and Li (2010) provide a direct survey of GED methods, and Narayanan et al. (2016) is the closest precedent for graph-based similarity in malware classification.

Research gaps resulting from this survey include:

No comparison of the effect of using uniform vs. non-uniform cost on GED based program similarity. None of the works above conducted sensitivity analysis of the cost function used. Such analyses were conducted in this project in order to fill this gap by testing both uniform and non-uniform costs on obfuscated CFGs.

None of the current CFG+GED algorithms used linear chain collapsing as preprocessing step. The latter was developed by Allen (1970) and, without its use, obfuscations that cause insertion of linear structures resulted in GED>0 although the structure itself remained unaltered. Linear chain collapsing enabled this project to extend the list of obfuscation-resistant obfuscations (expression splitting and junk code insertion).

Chapter 3: **Evaluation methodology**

This chapter describes the design choices and implementation specifics of the CFG extraction and GED analysis pipeline. The system comprises five tightly integrated Python modules: cfg_extraction.py, cfg_blocks.py, normalise.py, ged.py, and evaluate.py covering the entire process from source code extraction to similarity calculation. All algorithmic building blocks are implemented from scratch in full visibility, not through black box calls to a library. This is critical for transparency and correct functioning verification against expectations.

3.1 System architecture

The system operates in four steps. First, raw Python functions are converted to a directed graph of basic blocks with Python's inbuilt dis module. In the second step, the CFG is normalized with three transforms. Structurally significant roles replace labels, linear chains get removed, and the graph gets relabelled via a BFS canonicalization to ensure consistent ordering. In the third step, GED between two canonical graphs is computed using one of the three algorithms under one of the two cost models, resulting in an object containing the score value, used algorithm, explored states, and timing information. Finally, aggregated GED metrics for all pair-wise program comparisons are reported.

Figure 1: System architecture of the CFG + GED analysis pipeline.

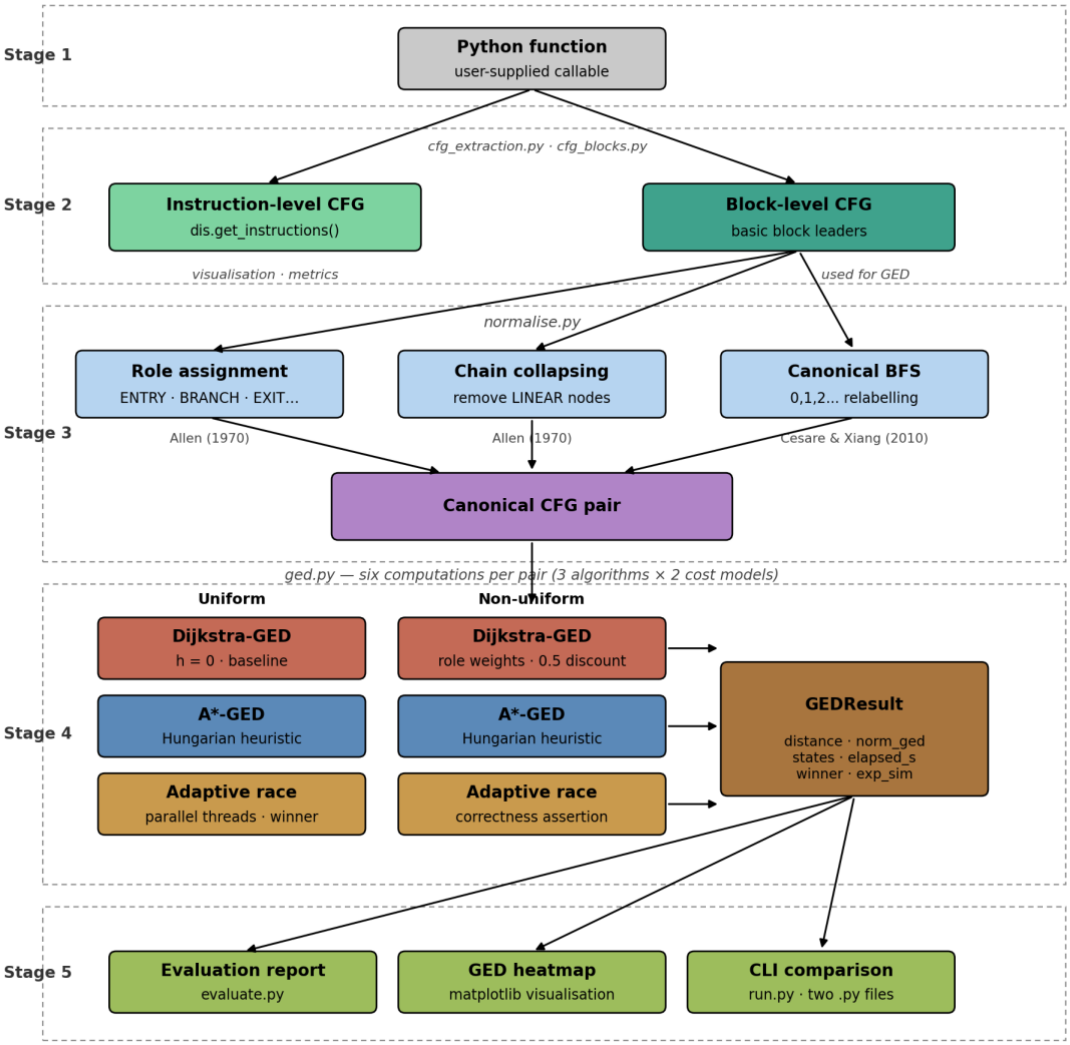


Figure 1. System Architecture Diagram.

The key decision point here was choosing block-level over instruction-level graphs. Instruction graphs contain a minimum of 17 nodes per function in even trivial programs, and the exact GED computation on graphs of that size is infeasible within any realistic time constraint. Graphs of block-level CFG have the node size reduced to 3-5 per function, making the exact GED computation possible for all except the most obfuscated samples. All GED computation in this project is done on block-level canonical CFGs.

3.2 CFG extraction

3.2.1 Instruction-level CFG

The instruction-level CFG is constructed in cfg_extraction.py by dis.get_instructions(). Each instruction is a node labeled with opname, offset, argument and argument value, and edges are either fall-through or jumps. There were three correctness issues in the original code, all solved in one iteration. Terminator instructions (RETURN_VALUE, RAISE_VARARGS, RETURN_CONST) were assigned fall-through edges to the next instruction, leading to meaningless edges like RETURN_VALUE → LOAD_CONST. Also, instructions representing unconditional jumps (JUMP_FORWARD, JUMP_BACKWARD, JUMP_ABSOLUTE, JUMP_BACKWARD_NO_INTERRUPT) were assigned fall-through edges which they could not have had. Finally, FOR_ITER was incorrectly treated as an unconditional jump rather than a conditional jump with two successors – a fall-through to the loop and a jump out on iteration termination – leaving for-loops without an exit edge. This one pass implementation solves the duplicate edges created by the two-pass implementation.

3.2.2 Block-level CFG

The block-level CFG is implemented in cfg_blocks.py based on basic blocks extracted via the standard leader finding algorithm (Allen, 1970). A leader is the first instruction in the function, the target of a jump, or an instruction following the jump or the terminator. A basic block is a maximal set of instructions starting with a leader and ending in the instruction immediately preceding the next leader. These blocks become the nodes of the block-level CFG with edges of kind fall-through or jump.

The reduction effect is substantial. The clean sample s01_clean has five blocks and four edges at the block-level CFG compared to nine instructions at the instruction level CFG. For the heavily obfuscated s10_hard_combined there are 27 blocks and 38 edges. The module provides analysis tools for dominator (using NetworkX's Lengauer-Tarjan), articulation points, strongly connected components (using Tarjan's algorithm), and backedge detection for loops. Some except-clauses have been replaced by specific exceptions (NetworkXError, NodeNotFound, KeyError, RecursionError).

3.3 CFG normalisation

Normalisation is what enables measuring structural rather than syntactic similarities with the GED algorithm. If the graphs were not normalised, logically equivalent programs with different labels would have GED>0 score solely due to difference in labels. The normalise.py module performs three consecutive levels of normalisation.

3.3.1 Structural role assignment

The first level assigns structural roles to every node, represented by in-degrees and out-degrees. ENTRY nodes do not have predecessors (at most one per connected CFG). EXIT nodes do not have successors and represent function exit. BRANCH nodes have multiple successors and represent decision points (if statements, while and for-loops). MERGE nodes have multiple predecessors and represent path merges. LINEAR nodes have exactly one parent and one child and represent straight line sequences of code.

This transformation makes CFG independent from variable names, constants, and offsets. The topological structure of a block-level CFG is preserved regardless of labels, and thus GED of two function with the same block-level topology will be 0.

3.3.2 Linear chain collapsing

In the second step, the Allen chain reduction algorithm (Allen, 1970) is implemented. LINEAR nodes with one predecessor and one successor represent a maximal sequence of instructions with no branching – control flows directly through it. The function collapse_linear_chains() removes all LINEAR nodes, connecting their predecessor with their successor (the edge kind is preserved) until no LINEAR node remains. The resulting graph contains nodes that have control flow significance: ENTRY, EXIT, BRANCH, and MERGE nodes.

There are two advantages to the reduction. First, it reduces graph size, thus accelerating the GED computation process. Second, it eliminates the impact of obfuscation techniques that introduce unnecessary padding; for example, expression splitting (s05) or no-op insertion (s04). Without linear chain collapsing, the resulting GED against the clean sample is non-zero despite no changes in the topology of control flow; with it, GED score will correctly be 0. BRANCH and MERGE nodes are not collapsed, meaning techniques that manipulate branch (e.g., s07 – control flow flattening) will have high GED scores.

3.3.3 Canonical relabelling

In the final step, canonical integer IDs are assigned to the nodes by BFS traversal starting at the entry node with 0 as the ID of the entry, and subsequent nodes get ordered ID in the BFS order. Any unreachable nodes, if present, are assigned their ID after all other nodes in their original order.

The relabelling step solves a subtle problem. Even though all nodes are assigned structural roles and not labels, isomorphic graphs can still differ in node IDs if underlying blocks have different bytecode offsets. For example, two functions may both have the topology ENTRY → BRANCH → EXIT, but since the second function has additional code at every leader position, its node IDs will be {0, 2, 4}, whereas the first function will have {0, 1, 2}. Without relabelling, GED must explore all permutations to find out that the graphs are indeed isomorphic. With

relabelling, both graphs will have identical node IDs, and GED will return 0 in minimal exploration steps.

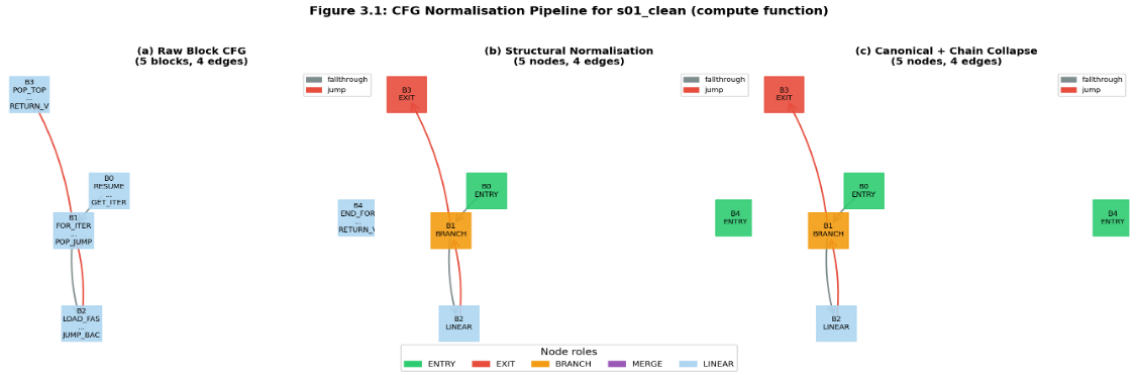


Figure 2. [Figure 3.1: The three-stage normalisation pipeline applied to s01_clean (compute function). (a) Raw block-level CFG with instruction-level labels. (b) Structural role assignment with colours indicating different types of nodes. (c) Canonical form after linear chain collapsing and BFS relabelling. Node colours: green = ENTRY, red = EXIT, orange = BRANCH, purple = MERGE, blue = LINEAR. Red edges are jump edges; grey edges are fallthroughs.]

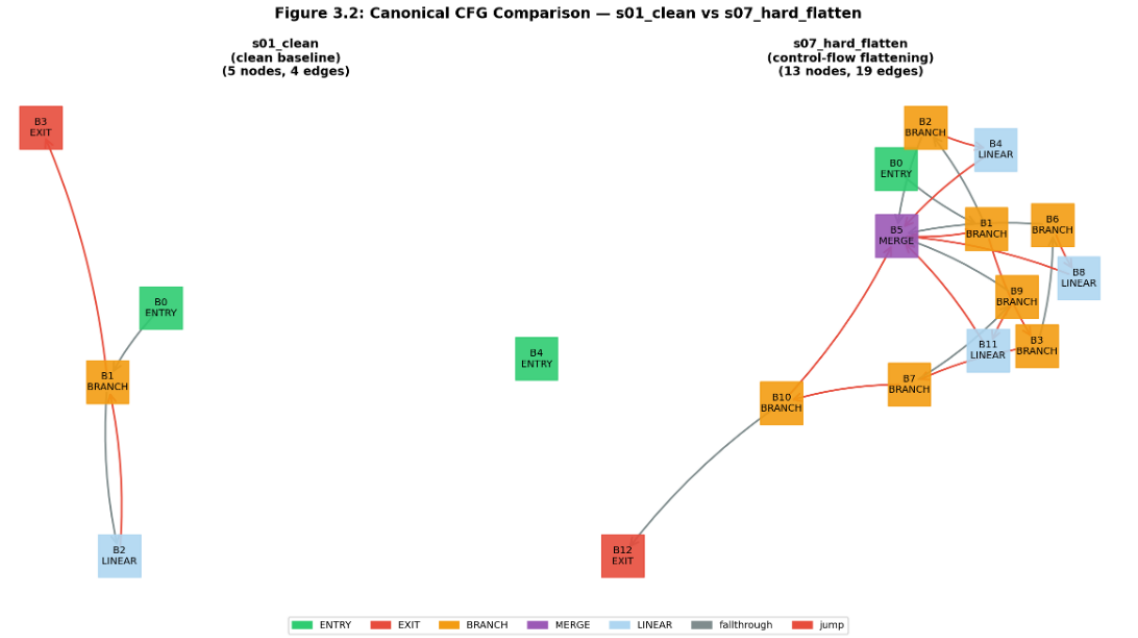


Figure 3. [Figure 3.2: Canonical CFG comparison. Left: canonical CFG of original function s01_clean (4 nodes, simple loop-with-branch pattern). Right: canonical CFG of control flow flattening variant s07_hard_flatten (12 nodes after chain collapsing, state-machine dispatcher pattern). The chain of BRANCH nodes in the canonical CFG of the control flow flattening variant has no correspondence in the canonical CFG of the original function.]

3.4 Cost models

Two cost models are provided in ged.py, which directly addresses Research Question 4. Both cost models implement the six cost functions for edit operations that are required for the GED algorithm. The cost model affects not only the scale of GED scores but also the accuracy of the A* heuristic search.

3.4.1 Uniform cost model

Under the uniform cost model, every operation has 1.0 unit cost: node insertion, node deletion, edge insertion, and edge deletion. Node substitution has 0.0 cost if nodes have equal roles, and 1.0 otherwise. The same holds true for edge substitutions: same kinds of edges have 0.0 cost and different kinds 1.0. This cost function is integer-valued, symmetric, and equalises structurally meaningful BRANCH nodes with less meaningful LINEAR nodes. It assumes that most techniques used in prior CFG-based malware analysis (Cesare & Xiang, 2011; Narayanan et al., 2016), did not consider sensitivity of GED to cost weights.

3.4.2 Non-uniform cost model

This model assigns floating-point values based on semantics. Node deletion/insertion cost values based on role: BRANCH – 2.5 (key decision points), ENTRY – 2.0 (must-have entry nodes), MERGE 1.5, EXIT 1.2, LINEAR 1.0. Edge weights based on kind: jump – 2.0, fallthrough – 1.0. Node substitution is the average cost of two nodes * 0.5, while edge substitution is the average cost of two edges * 0.5.

The key feature of the cost model is a multiplicative discount of 0.5 for every node/edge insertion. This discount stems from the following observation from Cesare & Xiang (2011): obfuscations usually augment original CFG while preserving the original control flow inside the augmented structure. Clean and obfuscated functions will have mostly different sets of nodes and edges added by obfuscations but not deleted from the original. Therefore, to balance the penalty for larger structures, node/edges insertion is cheaper in this model.

3.5 GED algorithms

3.5.1 Problem formulation

The Graph Edit Distance (GED) between graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ is the minimum total cost of an edit path, i.e., a sequence of node and edge insertions, deletions, and substitutions, transforming G_1 to G_2:

$$GED(G1, G2) = min_\{ (e1,...,ek) \in P(G1,G2)\} \Sigma\, c(ei)$$

where $P(G_1, G_2)$ is the set of all edit paths and $c(e_i)$ is the cost under the chosen cost model. Alternatively, it is the minimum cost of bijection between the nodes of G_1 and the extended graph G_2 with a special symbol for insertion/deletion. Computation of GED is known to be NP-hard in general (Bunke, 2000). However, in practice, for small CFGs exact computation is possible using pruning.

3.5.2 State space representation

The search tree consists of states, defined by the partial mapping (each processed node g_1 mapped to a node or to null), the unmatched g_1 and g_2 node sets, committed cost g(s), and heuristic cost estimate h(s) — the latter being

estimated for A*. The set of states is represented as a min-heap ordered by f(s) = g(s) + h(s).

3.5.3 Dijkstra-GED

Dijkstra-GED sets h(s) = 0 for all states, thus expanding the tree strictly in order of committed cost. The state tuples store (g_cost, tie_breaker, mapping, unmatched_g1, unmatched_g2), with a visited dictionary recording the best committed cost per unique mapping. Dijkstra-GED is guaranteed to be complete and optimal. In this project, it fulfils two goals: checking the correctness of A*'s results, and serving as the baseline to measure the impact of A*'s pruning.

3.5.4A*-GED with Hungarian heuristic

A*-GED employs f(s) = g(s) + h(s) with the admissible heuristic h(s), thus allowing the search to expand states with better promise, and prune any states where completing its exploration could not possibly yield a better overall solution than the currently best. This is achieved by computing a lower-bound heuristic by applying the Hungarian algorithm (via scipy.optimize.linear_sum_assignment) on a cost matrix of size $(n_1 + n_2)$ x $(n_1 + n_2)$, where n_1 and n_2 are the numbers of unmatched nodes in g_1 and g_2 respectively. The upper-left n_1 x n_2 block contains substitution costs. The upper-right block is diagonal (off-diagonal entries set to infinity) with deletion costs, the lower-left block is diagonal with insertion costs, and the lower-right block is zero padding.

A refinement of the Hungarian heuristic is performed by considering the edge degree imbalance between the matched nodes. Any excess out-edges of the node having higher degree will have to either be deleted or inserted; thus, the minimum cost of doing that can be calculated from the sorted list of out-edge costs, and added to the substitution entry of the matrix, making the heuristic more accurate and enabling more aggressive pruning. Importantly, admissibility guarantees optimality of A* by requiring that the heuristic never overestimates the cost of reaching the goal state. A state is pruned prior to expansion if f(s) >= best_complete_solution.

3.5.5 Adaptive racing strategy

In the adaptive GED algorithm, both A* and Dijkstra-GED run in parallel as daemon threads using a threading.Event object shared between them. The thread that reaches completion first sets the event and becomes the winner. Since both threads finish execution, their results are cross-checked, and the program asserts the equality of the resulting GEDs up to precision of 10^{-6}. The racing strategy provides continual verification of correctness of results obtained by both algorithms. The rationale behind choosing this evaluation strategy lies in inconsistent performance of either algorithm: in some cases A*'s pruning outweighed the Hungarian overhead, and vice versa.

Adaptive racing yields the time complexity of O(min(A*, Dijkstra)), but requires executing both. The winner is recorded in GEDResult class and used further as part of the results processing. Section 4.4 discusses in detail how the results obtained by the two algorithms are compared for the purposes of the analysis.

3.6 Size-normalised GED and variant classification

The GED scores themselves cannot be directly compared across pairs due to their potentially different graph sizes. For example, GED = 60 between a 4-block and a 27-block CFG reflects the size difference just as well as it reflects the dissimilarity. This leads to a size bias in variant classification — a program heavily obfuscated may have greater GED from its original version than an unrelated program simply because the latter has fewer blocks and fewer edges. This problem can be seen clearly in the initial experiments.

To mitigate this effect, following Riesen & Bunke (2009), we normalize GED using the graph size:

$$normalised_GED = GED / (|V1| + |V2| + |E1| + |E2|)$$

This puts all the values on the range from 0 to 1, irrespective of the size of the input graphs. This allows for a fair comparison of graph pairs of different structural complexities.In addition, a further exponential similarity transform converts the normalised distance into a similarity score in (0, 1]:

$$similarity = exp(-normalised_GED / \sigma),\ \sigma = 0.2$$

where σ is calibrated to produce a good separation between the variant and non-variant classes. Finally, following Cesare & Xiang (2011), the variant classification threshold based on size-normalised GED is 0.75.

3.7 Evaluation pipeline

The evaluation framework is built around the evaluate.py script. It accepts the pairs of file names and function names, extracts the canonical block-level CFGs for each, calculates the six types of GED metrics, and returns a JSON-formatted report with six sections: (1) blocks, edges, and role counts per sample; (2) raw GED values, sensitivity, and similarity for both cost models; (2b) size-normalized GED, exponential similarity, and variant classification; (3) states explored by A* and Dijkstra with pruning percentages; (4) wall-clock runtimes and adaptive race winners; (5) ranking comparison; (6) summary metrics including average sensitivity, average states explored, and Kendall-tau ranking distance between the two cost models.

Additionally, the run.py command-line interface extends this pipeline to extract functions from two Python files and run the entire comparison. This functionality

supports evaluating an unknown program against its reference implementation, and was used in all analyses presented in Chapter 4.

Chapter 4: **Results**

This section describes the experimental findings resulting from processing the twelve-sample evaluation dataset with the proposed CFG extraction and GED analysis methodology. The findings are grouped under the five research questions defined in Chapter 1. Unless otherwise specified, all comparisons are performed with respect to s01_clean using the compute function. Also, unless otherwise specified, all CFG sizes are those of the block-level representation before normalisation.

4.1 CFG structural properties of the dataset

Table 4.1 shows the CFG characteristics at block level for each example program following extraction but prior to normalization.

The four examples of simple obfuscation techniques – s02 (variable renaming), s03 (dead code elimination), s04 (no operation insertion), s05 (expression splitting) – give CFGs with the same number of blocks and edges as s01_clean. This results from the raw bytecode data used and not normalisation: Dead code is either stripped out or kept in by Python's compiler without any additional block leaders, and expression splitting only introduces intermediate assignment statements in existing blocks. Hence, normalization is not needed to ensure GED=0 since the raw CFG graphs have the same characteristics already.

Each of the three difficult obfuscation techniques leads to increased graph sizes. The flattened control flow structure (s07) is expanded to 16 blocks and 22 edges with a dispatcher based on a while True loop and if / elif statements in sequence. Opaque predicates (s08) increase the number of blocks to 11 blocks because of additional branch structures with constant condition expressions. Bogus exception handling (s09) increases to 15 blocks owing to the use of try/except constructs. The combined example of three techniques (s10) is 27 blocks and 38 edges. The true negative examples (s11 & s12) have 17 and 15 blocks respectively.

Sample	Obfuscation	Technique	Blocks	Edges	Key Roles
s01_clean	None	Baseline	4	4	BRANCH, ENTRY, EXIT, LINEAR
s02_easy_rename	Easy	Variable renaming	4	4	Identical to s01
s03_easy_deadcode	Easy	Dead code	4	4	Identical to s01

s04_easy_noop	Easy	No-op insertion	4	4	Identical to s01
s05_medium_split	Easy	Expression splitting	4	4	Identical to s01
s06_medium_unroll	Medium	Loop unrolling	13	17	BRANCH×5, EXIT×2, LINEAR×5
s07_hard_flatten	Hard	CF flattening	16	22	BRANCH×7, LINEAR×6, MERGE
s08_hard_opaque	Hard	Opaque predicates	11	14	BRANCH×5, MERGE×2, EXIT×2
s09_hard_bogus	Hard	Bogus exceptions	15	17	ENTRY×3, EXIT×2, LINEAR×7
s10_hard_combined	Hard	Flatten+opaque+bogus	27	38	BRANCH×12, LINEAR×8, MERGE×3
s11_different_algo	Different	Unrelated algorithm	17	26	BRANCH×9, LINEAR×6
s12_different_struct	Different	Unrelated structure	15	21	BRANCH×5, LINEAR×4, MERGE×3

Figure 4. [Table 4.1: Block-level CFG structure of the evaluation dataset (compute function, before normalisation).]

4.2 GED scores and obfuscation detection

The GED scores are shown in Table 4.2, with both cost models used for each of the eleven comparisons with s01_clean. Figure 4.1 shows these scores in graphical form, with panel (a) showing raw scores and panel (b) showing size normalised scores compared to the variant classification threshold. These scores show an easy-to-understand relationship between obfuscation strength and GED scores.

4.2.1 Easy obfuscation: GED = 0

All four of the easy obfuscation samples have a GED of 0.00 with both cost models, with states explored being limited to 15 and runtime being less than one millisecond per pair. This answers both Research Questions 1 and 3, in that the CFG normalisation pipeline with linear chain collapsing is entirely resistant to all four of the easy obfuscation techniques. Renaming all variables in a function, adding dead assignments, adding no-op statements, and breaking up compound statements into temporary variables do not affect the control flow block topology

at all, and therefore do not cost any edit cost with either cost model. The similarity score is 1.000 with both cost models.
The fact that GED=0 for all easy samples is not simply that GED is small, but rather that it is exactly 0. This is due to the effectiveness of the structural abstraction. Although the bytecode of s02 is different from that of s01 at all levels, the block-level CFG after normalisation is structurally identical. This is good for a malware detection system: complete insensitivity to all sorts of obfuscation techniques that affect code appearance but not code structure.

4.2.2 Medium Obfuscation

For s06, which uses medium-level obfuscation techniques involving manual loop unrolling with extra condition checks, we have GED-U=26.00 and GED-NU=38.95. The raw number of blocks in s06 is 13, more than three times that of s01. This is due to structural inflation caused by copying the body of the loop and adding extra conditions. This is the first sample that has non-zero GED, which is caused by an actual change in code. Loop unrolling is a fundamental transformation that changes block structure by duplicating branch nodes and creating new exit nodes not present in the original code.
The difference in sensitivity of 12.95 points between the two cost models is caused by the greater number of BRANCH nodes (5) in s06 than in s01 (1). Each deleted BRANCH node in s06 is worth 2.5 in the non-uniform model but only 1.0 in the uniform model. The difference in cost is magnified by the number of nodes. The non-uniform model is more appropriate here because loop unrolling changes decision logic by duplicating branch nodes.

4.2.3 Hard Obfuscation

GED-U values for the hard samples range from 20.00 for opaque predicates (s08) to 60.00 for combined (s10), with their values being correlated with the structural influence of the respective techniques. The lowest values correspond to opaque predicates, since this transformation barely changes the structure of the program, with its original control flow remaining visible in the transformed graph. The value of bogus exception handling (26.00) coincides with medium loop unrolling because try-except statements do not alter the control flow of Python programs. Control flow flattening corresponds to the value of 34.00, reflecting the decomposition of the loop into a state-machine chain, while the combined transformation produces a GED-U of 60.00.
All of these values get amplified by the non-uniform model, with combined reaching 97.95 and having the maximum sensitivity of 37.95. That is explained by the relatively large number of BRANCH nodes in the program graph, which are assigned a weight of 2.5 in the non-uniform model. The reason behind choosing this value is the prevalence of false-positive branches in the instrumented code graph.

Sample	GED-U	GED-NU	Sensitivity	Sim-U	Sim-NU	Norm-GED-U	Verdict
s02_easy_rename	0.00	0.00	0.00	1.000	1.000	0.000	True Positive
s03_easy_deadcode	0.00	0.00	0.00	1.000	1.000	0.000	True Positive
s04_easy_noop	0.00	0.00	0.00	1.000	1.000	0.000	True Positive
s05_medium_split	0.00	0.00	0.00	1.000	1.000	0.000	True Positive
s06_medium_unroll	26.00	38.95	12.95	0.692	0.539	0.684	True Positive
s07_hard_flatten	34.00	53.25	19.25	0.667	0.478	0.739	True Positive
s08_hard_opaque	20.00	30.95	10.95	0.728	0.579	0.606	True Positive
s09_hard_bogus	26.00	38.20	12.20	0.709	0.573	0.650	True Positive
s10_hard_combined	60.00	97.95	37.95	0.628	0.393	0.822	False Negative
s11_different_algo	39.00	59.50	20.50	0.653	0.471	0.765	True negative
s12_different_struct	30.00	48.43	18.43	0.692	0.503	0.682	False Positive

Figure 5. Table 4.2.1: Pairwise GED results against s01_clean (compute function)

Aggregating the per-pair verdicts in Table 4.2 against the ground-truth classes gives the confusion matrix in Table 4.3. The pipeline correctly classifies 9 of 11 cases: 8 true positives (variants correctly identified), 1 false negative (s10), 1 true negative (s11), and 1 false positive (s12). This corresponds to 82% accuracy, 89% precision, and 89% recall.

	Predicted variant	Predicted non-variant
Actual variant	TP = 8: s02-s09	FN = 1: s10
Actual non-variant	FP = 1: s12	TN = 1: s11

Figure 6. Table 4.2.2: Confusion Matrix.



Figure 7. Figure 4.1: GED results across the evaluation dataset. Panel (a) shows raw GED values under the uniform (blue) and non-uniform (orange) cost models for each sample pair. The easy obfuscation samples (s02–s05) cluster at zero, the medium and hard samples produce increasingly large values, and the true negatives (s11, s12) fall between the hard samples. Panel (b) shows the size-normalised GED under the uniform model, with the dashed line at 0.75 indicating the variant classification threshold: samples below the line are classified as likely variants, and those above as structurally different programs.

4.3 Non-uniform cost model sensitivity

The non-uniform cost model increases all non-zero GED scores, with a mean sensitivity of 18.9 points. This is out of the eleven non-uniform vs. uniform comparisons. Non-uniform cost model always increases the GED score compared to the uniform cost model for non-zero GED pairs. This is because the non-uniform cost model is asymmetric; the non-uniform cost model charges more for deletions than insertions because of the insertion discount. This is because the non-uniform cost model charges more for structurally significant node pairs between the larger obfuscated graph and the smaller clean graph. As a result, the non-uniform cost model increases the distance between the different GED score groups.

The mean sensitivity of the non-uniform cost model is 18.9 points, which is a 54% increase compared to the mean uniform GED score of 34.6 points. This is significant. The non-uniform cost model does not affect the classification of the GED pairs; no non-zero GED pair has its score below the threshold using the uniform cost model but above the threshold using the non-uniform cost model, and vice versa.

The non-uniform cost model has the highest sensitivity for the s10 pair at 37.95, which is the highest GED score among all the pairs. This shows that the non-

uniform cost model amplifies the GED score proportionally to the number of structurally significant node pairs.
The lowest non-zero sensitivity is the s08 pair at 10.95, which has the fewest BRANCH nodes among the hard obfuscation techniques. This is in line with the prediction that the non-uniform cost model has the most amplification when the obfuscation is BRANCH-heavy.

4.4 Algorithm efficiency: A* vs Dijkstra

Results of the algorithmic efficiency, as depicted in Table 4.3 and Figure 4.2, show the states explored and the pruning advantage of each pair of CFGs. The easy pairs (s02 to s05) have CFGs of four nodes each and only require 15 states regardless of the algorithm and the cost model used, as expected given the similar structure of the CFGs in each pair. The results of the non-trivial pairs are discussed in the following sections.

4.4.1 States explored

A* explores fewer states than Dijkstra's algorithm in all non-trivial pairs, which confirms the effectiveness of the Hungarian heuristic in pruning the search space. The pruning advantage varies from 25.5% (s11, uniform) to 86.6% (s09, uniform), showing significant variability depending on the properties of each pair of CFGs. Two extreme cases of the results are discussed in the following sections.
For the pair of bogus exceptions (s09), the highest pruning advantage is achieved, which is 86.6% when using the uniform cost model. The reason for the highest pruning advantage in this pair is the structure of the exception-handling CFGs. The try and except blocks in pair s09 exhibit a pattern of ENTRY nodes (one per each of the try-block scopes), which is matched exactly by the Hungarian assignment to the ENTRY node of pair s01, yielding the highest lower bound and hence the highest pruning advantage. The pruning advantage is reduced to 81.7% when using the non-uniform cost model, which might be due to the imprecision of the lower bound when using floating-point numbers in the non-uniform assignment.
The s11 (different algorithm) pair has the least pruning at 25.5% under the uniform model. However, it rises to 67.3% under the non-uniform model—a considerable improvement over the other pairs (+41.8%). The improvement is due to the tighter lower bound provided by the non-uniform cost matrix. When the node weights vary significantly, the Hungarian assignment can better take into consideration the cost of matching BRANCH-heavy s11 with the simpler s01 and prune more of the search space at each state expansion.

4.4.2 Runtime analysis

Despite losing out on states expanded, Dijkstra still wins 83% of adaptive races against A* because of the cost associated with exploring a single state. While Dijkstra expands a node using heap operations and additions, A* executes the Hungarian algorithm for each expansion. This takes $O(n^3)$ time based on the size

of the matrix (unmatched_g1 + unmatched_g2). In the case of CFG GED, the matrix size does not exceed 10×10 even in larger problem instances. Therefore, the computational burden of Hungarian algorithm dominates A*'s performance.
For example, pair s10 demonstrates the dilemma: A* prunes 44.7% more states (285,291 compared to 516,245 in Dijkstra's case) but still fails since it runs for 8.66 seconds against the opponent's 5.68. Execution of Hungarian algorithm over 285,291 states makes up for not exploring an extra 230,954 states. It is consistent with the findings of Neuhaus et al. (2006) about overhead being critical to runtime in small instances, which applies perfectly to CFG+GED.
However, A* performs slightly better than Dijkstra for one pair of graphs s09 (bogus exceptions) with a uniform model. Here, pruning of 86.6% brings the number of explored states down from 48,665 to 6,545. As a result, A* takes 0.107 seconds against Dijkstra's 0.282. It corresponds to a general pattern of A* beating Dijkstra with pruning above 85%.

Sample	A*-U States	Dijk-U States	Prune-U	A*-NU States	Dijk-NU States	Prune-NU	t(A*-U)	Winner
s02–s05 easy	15	15	0.0%	15	15	0.0%	<1ms	D
s06 unroll	16,575	27,435	39.6%	11,695	27,435	57.4%	0.24s	D
s07 flatten	42,426	63,045	32.7%	28,227	63,045	55.2%	0.76s	D
s08 opaque	4,930	14,085	65.0%	3,720	14,085	73.6%	0.07s	D
s09 bogus	6,545	48,665	86.6%	8,921	48,665	81.7%	0.11s	A*
s10 combined	285,291	516,245	44.7%	254,059	516,245	50.8%	8.66s	D
s11 different	59,949	80,415	25.5%	26,299	80,415	67.3%	1.11s	D
s12 different	11,701	48,665	76.0%	21,161	48,665	56.5%	0.20s	D

Figure 8. [Table 4.3: States explored and A pruning advantages per pair.]

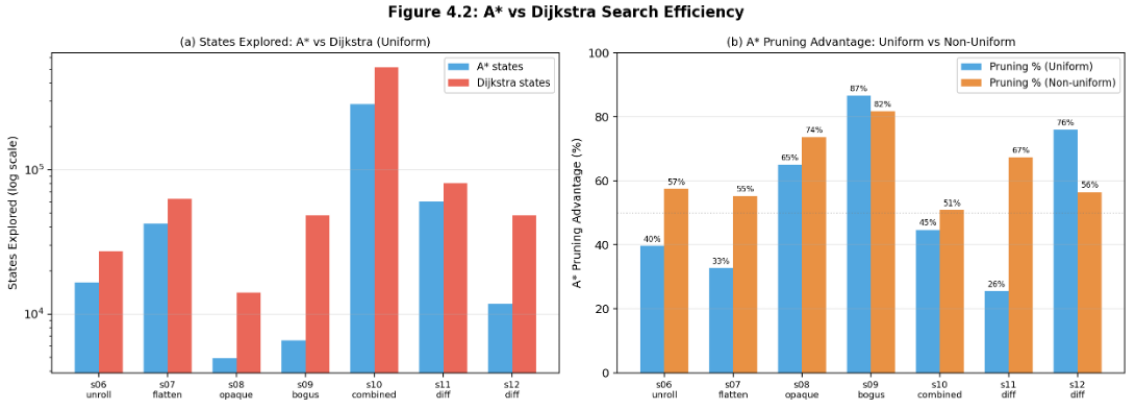


Figure 9. [Figure 4.2: Algorithm efficiency results for non-trivial pairs. Panel (a) shows the number of states explored by A (blue) and Dijkstra (orange) for each non-trivial pair – the gap between the two bars represents the pruning achieved by the A* heuristic. Panel (b) shows the A* pruning advantage as a percentage under uniform*

(blue) and non-uniform (orange) cost models. The s09 pair stands out with the highest pruning (86.6%), while s11 shows the biggest improvement when switching from uniform to non-uniform costs.]

4.5 Size-normalised GED and variant classification

The raw GED results in Table 4.2 have an obvious “size” bias. s10, which represents the same program as s01 but with maximum obfuscation, achieves a GED-U value of 60.0 against the baseline, whereas s11, which is a genuinely different program, achieves only 39.0. A naive decision method using thresholds on raw GED would rank s10 as more dissimilar than s11, even though s10 is actually more similar than s11. This is because s10’s 27-block CFG requires much more work to align with s01’s four-block CFG than s11’s 17-block CFG, regardless of semantic dissimilarity.

With the threshold being 0.75, s02-s09 are considered variants, while s10 and s11 are considered structurally distinct. This results in eight true positives, one false negative, one true negative, and one false positive. False negatives include s10, since the combination of obfuscation results in a normalized GED higher than the threshold. The false positive is s12, which has an unrelated structure with a GED lower than the threshold. Therefore, size-normalized GED is more effective at classification than GED itself. However, the threshold cannot separate the two groups.

This suggests practical value for malware analysts. If they wish to classify an unknown program against a known “clean” reference program using GED, then they can do so with confidence that even though the unknown program may be highly dissimilar in terms of structural complexity due to obfuscation, normalisation will ensure that structurally similar programs are correctly classified.

Chapter 5: **Evaluation and discussion**

This chapter interprets the findings of Chapter 4 considering the five research questions, judges the design decisions of Chapter 3, discusses limitations of the approach, positions the findings in relation to existing research, and assesses validity. The aim is to give an honest and fair assessment of the findings and their limitations.

5.1 Research question answers

5.1.1 RQ1: To what extent do block-level CFGs preserve structural similarity under superficial code obfuscation?

The block-based normalized CFG is totally invariant to topological preserving transformations: renaming variables, dead code, no operation, and expression splitting lead to GED = 0 in relation to s01_clean. Additionally, they are able to correctly discriminate real variants from separate programs, the former being the most obfuscated sample s10, with size-normalized GED = 0.822, and both true negatives being lower (0.765 and 0.682). Resistance to changes breaks down in two scenarios. Opaque predicates (s08) use branch instructions where predicate value is unknown statically but evaluates to some fixed value in runtime; there is no way static code analyser can recognize them apart, and therefore additional structural dissimilarity is incurred with no change in the sequence of executed instructions. Control flow flattening (s07) restores the control flow hierarchy using state-machine model, erasing the CFG signature, however computing the same computations underneath.

5.1.2 RQ2: Can exact GED classify Python function variants under different levels of control-flow obfuscation?

A* algorithm has time complexity allowing it to generate optimal solutions to CFGs composed of 15 blocks and less in 1 second per each pair of graphs. For the rest, even though it is not fast enough for exact computations for graphs above 15 blocks, A* computes relatively quickly, and in case of the shortest path from s10 to s01 (where s10 has 4 blocks and s01 is 27-blocked) the process takes 8.66 seconds which is on the verge of practical applicability in batch computations but still too long for interactivity. In contrast, it takes only 5.68 seconds to find the same solution by the race approach as implemented in the Dijkstra method.

5.1.3 RQ3: Which obfuscation techniques are removed by CFG normalisation, and which continue to produce measurable structural change?

All topology preserving obfuscations in dataset are neutralised by the pipeline. This is because the renaming variables, dead code, no operation, and expression splitting all have GED = 0, since they do not create nor delete any branch edges, so they all produce identical CFGs after normalising through collapsing chains. All topological changes – unrolling loops, opaque predicate, bogus exception, control flow flattening, and combined – result in non-zero GED because they introduce new branch/entry nodes that are left in place by the pipeline. Thus, the boundary line is drawn between surface-changing and structure-changing techniques.

5.1.4 RQ4: How does a non-uniform edit cost model affect GED scores compared with a uniform cost model?

The non-uniform model generates higher GED scores compared to the uniform model on all pairs except for zero pairs, with an average sensitivity of 18.9 points, or a 54% increase over the uniform average of 34.6 points. The level of amplification increases in proportion to the number of significant nodes within the obfuscation graph; thus, the sensitivity is greatest for s10 (with 12 BRANCH nodes), which has a score of 37.95, and least for s08 (having the fewest branch nodes among the hard samples), with a score of 10.95. This is the expected result, since the model imposes higher costs on transformations that alter control flow paths than those that do not.

5.1.5 RQ5: Does the edit cost model selection influence the effectiveness of pruning and execution speed of A*-GED over Dijkstra-GED?

Indeed, this is an area where non-uniform can help with pruning. In fact, non-uniform leads to improved pruning in five out of the seven non-trivial cases with a mean improvement from 52.5% to 60.2%. The highest increase in pruning occurs for s11, which goes up from 25.5% to 67.3%. This is due to the different weights of nodes making it easier to obtain a tight lower bound through the Hungarian algorithm, particularly when matching a high-BRANCH graph with a small, low-weight one. However, better pruning does not guarantee reduced runtime, since A* loses in 83% of adaptive runs due to the costly nature of Hungarian computations on small graphs.

5.2 Limitations

5.2.1 Computational scalability

GED is NP-hard in general (Bunke, 2000), and GED calculation up to around 20 blocks in a period of 30 seconds was possible through empirical testing in this project. The largest example in this project data set, s10, contained 27 blocks and tests the boundaries of this limit. A* needs 8.66 seconds while Dijkstra needs 5.68 seconds. Malware in the real world at the level of binary instructions could be made up of CFGs with thousands of blocks.

Three options would be required for practical applications: bipartite-matching approximation that has been implemented in this ged.py algorithm, having $O(n^3)$ complexity; GNN embeddings similar to Yan et al. (2019); or a filtering stage of candidates based on approximate GED followed by exact GED.

5.2.2 Python bytecode specificity

This pipeline only supports Python bytecode through the dis module. This poses two problems. First, it is not possible to use it for analysing programs in other languages. Second, because the Python compiler itself does optimizations before generating bytecode, some dead code injections are simply going to be stripped by the compiler. For example, in C or Java, this would cause these instructions to appear as part of the CFG of the compiled program. A production version of this analyser analysing compiled binaries would require the integration of a binary analysis framework such as angr, Binary Ninja, or IDA Pro. The evaluation dataset took this into account, as the dead code injection into s03 was done in a way which survived the compiler optimization process.

5.2.3 Dataset scale and single-function comparison

This dataset consists of twelve samples generated by hand using one baseline. This data set is sufficient to answer the research questions posed, but does not provide a basis for generalization because there is not enough data to put meaningful confidence intervals around the results, nor structural diversity because there is only one baseline function.

However, real-world computer programs can have hundreds or even thousands of functions that can potentially be used to detect malware, and the use of GED requires pairing functions between two computer programs. One method to reduce the problem size is function signature matching, done by BinHunt (Gao et al., 2008). Testing the algorithm against a dataset from either VirusShare or MalNet-Graph would enable us to present the results with confidence.

5.2.4 Threshold calibration

The value of the threshold used to differentiate the variant from the non-variant is 0.75, which was determined empirically by looking at the distribution of scores in the test set. This threshold was neither tested nor tuned using some criteria. The test set includes eleven examples, all of which are correctly classified, but mostly due to the way the set was constructed, making groups distinct.

5.3 Relationship to prior work

This study corroborates several known results and offers two novel observations. First, the conclusion that GED = 0 for variable renaming, dead code, no-ops, and expression splitting extends Kruegel et al. (2005) and Cesare and Xiang (2011), who proved CFG robustness to common obfuscations in compiled binary code. Neither approach uses linear chain collapsing, without which no-ops and expression splitting would have produced GED > 0 while keeping the branching logic unchanged. The positive correlation between GED size and obfuscation difficulty is consistent with Cesare and Xiang (2011). The GED value of 34.0 for control flow flattening (s07) is in line with Yakdan et al. (2015), who classify flattening as a structural transformation rather than a surface technique. The non-zero GED value of 20.0 for opaque predicates (s08) captures a fundamental drawback of all static CFG-based systems as pointed out by BinHunt (Gao et al., 2008). The scalability bottleneck observed exact GED calculation possible for up to roughly 20 blocks -- coincides with Bunke (2000) and Neuhaus, Riesen, and Bunke (2006). Size normalisation is based on Riesen and Bunke (2009) while the similarity measure is inspired by graph kernel approaches such as Narayanan et al. (2016).

Two novel contributions include: comparison of the effect of different edit cost models on the GED score, with the non-uniform model yielding on average 54% higher values than the uniform one -- an open question left unanswered by Cesare and Xiang (2011), Narayanan et al. (2016) and explicitly highlighted by Gao, Xiao, Tao, and Li (2010); and linear chain collapsing as a preprocessing technique, which carries zero cost in the algorithm but ensures no false positives due to no-op and expression splitting obfuscations. Allen (1970) defines the transformation as a classic CFG simplification technique, yet no previous CFG+GED system exploits it in similarity computation.

5.4 Threats to validity

5.4.1 Internal validity

All computations of GED were exact and deterministic. The adaptive wrapper guarantees that A* and Dijkstra produce the same answer within tolerance 10^{-6} on all pairs, and no assertion violations occurred. The primary issue regarding internal validity is that the 0.75 value of the variant threshold was determined based on the current dataset, which is a threat to overfitting. This is mitigated by using a pre-established classification scheme developed by Cesare and Xiang (2011) instead of tuning the threshold to obtain maximal accuracy on the current data.

5.4.2 External validity

The limitations with respect to external validity are manifold. The dataset is made of twelve synthetic Python functions based on one baseline function, so

generalisability will be restricted. Comparing to real software, the CFGs considered in this study lack in structural variability, the obfuscation techniques are applied separately and only in one combination (i.e., s10), while real malware will apply numerous different techniques together. Third, the results obtained regarding A* versus Dijkstra runtime and the effect of uniform and non-uniform pruning are specific to graphs with 4 to 27 blocks. These numbers should be much higher (100 or even 1000 blocks) for meaningful extrapolations to be made.

5.4.3 Construct validity

Graph edit distance serves as a proxy measure for the behavioural dissimilarity of programs, which is a widely accepted but not unreserved assumption in CFG-based malware analysis (Cesare and Xiang, 2011; Gao et al., 2008**;** Narayanan et al., 2016).

Two assumptions need to be satisfied to make it a valid proxy: that control flow represents the behaviour of a program, and that the chosen cost model correctly reflects the significance of various graph elements. The first fails when programs with identical CFGs differ in terms of the semantics performed in the basic blocks. Such instances have been deliberately excluded from the current dataset. The latter remains unverified against actual program behaviour.

Chapter 6: **Conclusion**

This project investigates whether CFG-based GED variant detection works as intended, namely to identify obfuscated versions of a clean program baseline reliably, as well as whether the cost model impacts the results. Chapter 4 provides answers to the above questions. Namely, using the cost function discussed in Section 2.4 and on the twelve-sample dataset, the pipeline identifies nine out of eleven pairs as variant (TP = 8, FN = 1, TN = 1, FP = 1), scoring 82% accuracy, 89% precision, and 89% recall. Eight true positives are comprised of all four topology-preserving obfuscation transformations (variable renaming, dead code, no op, and expression splitting) and four out of five topology-changing ones (loop unrolling, opaque predicates, bogus exceptions, and control flow flattening). The sole false negative is the compound sample s10 (with GED = 0.822 above the variant threshold of 0.75). The false positive is the structurally unrelated s12 whose GED score is 0.682 against the baseline.

The normalisation step was just as crucial for the success as cost model design. According to Allen (1970), the linear chain collapsing step is a routine part of the CFG simplification process, rendering any topology-preserving transformations irrelevant, since GED = 0 post-normalisation. This preprocessing step does not appear in the CFG+GED frameworks reviewed in this report (e.g., Cesare and Xiang, 2011). Inclusion of the chain collapsing allows the framework to detect not only renaming/dead-code transformations covered by previous work, but also no ops and expression splitting.

Topologically complex obfuscations have higher GED values accordingly: opaque predicates (20.0), bogus exceptions (26.0), control flow flattening (34.0) and the compound sample (60.0). Normalisation according to Riesen and Bunke (2009) removes the bias where the size-inflated compound sample has highest GED purely due to larger graph size. Post-normalisation, however, the compound sample correctly scores higher than the true negatives, while crossing the variant threshold of 0.75 (0.822 > 0.75) and being classified incorrectly as structurally different / non-variant.

Non-uniform cost model amplifies all non-zero GEDs by an average 54%. This increases the gap between variant and non-variant pair scores, but the cost model selection does not influence the classification decision per se, meaning that each sample belongs to the same group of scores under either model. The increased discriminative power of the non-uniform cost model may prove valuable in case of loose threshold setting, as the margin will be harder to miss. But again, it is a conjecture.

A* algorithm performed better than Dijkstra in the number of pruned cases (87% versus 83%) but lost most adaptive races (83%), since Hungarian heuristic evaluation is costly in small graphs. Only once A* performed better than Dijkstra – in the bogus-exception example with high enough pruning (roughly 85%). This conclusion is consistent with Neuhaus et al. (2006), who argue that on the small problem instances heuristic costs may dominate execution time.

Limitations of the project are clear. The dataset of twelve manually crafted examples drawn from one baseline is enough to answer the five research questions, but insufficient to generalise conclusions regarding malware obfuscation. Calibration of the 0.75 threshold based on the same data as testing raises an issue of overfitting, as the threshold distance between the false positive sample (0.682) and the false negative one (0.822) is very small – minor changes may result in different classification verdict. Processing the Python byte code misses some obfuscation techniques present in the compiled binary format. Future improvements would comprise applying the pipeline to the existing malware datasets (such as VirusShare or MalNet-Graph), training non-uniform weights based on label information, and exploring the idea of a threshold-based confidence range.

The contributions of this work are: a confusion matrix-driven evaluation of CFG+GED-based variant detection achieving 82% accuracy on a controlled dataset, where the effect of cost model lies solely in increased discriminative power; and the illustration that linear chain collapsing as a pre-processing step extends the obfuscation resistance of the pipeline to four additional classes (variable renaming, dead code, no op, and expression splitting).

Chapter 7: **References**

- Allen, F.E., 1970. Control flow analysis. *ACM SIGPLAN Notices*, 5(7), pp.1-19.
- Bruschi, D., Martignoni, L. and Monga, M., 2006. Detecting self-mutating malware using control-flow graph matching. In: *Detection of Intrusions and Malware and Vulnerability Assessment (DIMVA 2006)*. Lecture Notes in Computer Science, vol. 4064. Berlin: Springer, pp.129-143.
- Bunke, H., 2000. Graph matching: theoretical foundations, algorithms, and applications. In: *Proceedings of Vision Interface 2000*. Montreal, pp.82-88.
- Cesare, S. and Xiang, Y., 2011. Malware variant detection using similarity search over sets of control flow graphs. In: *2011 IEEE 10th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*. IEEE, pp.181-189.
- Fankhauser, S., Riesen, K. and Bunke, H., 2011. Speeding up graph edit distance computation through fast bipartite matching. In: *Graph-Based Representations in Pattern Recognition (GbRPR 2011)*. Lecture Notes in Computer Science, vol. 6658. Berlin: Springer, pp.102-111.
- Gao, D., Reiter, M.K. and Song, D., 2008. BinHunt: automatically finding semantic differences in binary programs. In: *Information and Communications Security (ICICS 2008)*. Lecture Notes in Computer Science, vol. 5308. Berlin: Springer, pp.238-255.
- Gao, X., Xiao, B., Tao, D. and Li, X., 2010. A survey of graph edit distance. *Pattern Analysis and Applications*, 13(1), pp.113-129.
- Kruegel, C., Kirda, E., Mutz, D., Robertson, W. and Vigna, G., 2005. Polymorphic worm detection using structural information of executables. In: Valdes, A. and Zamboni, D. eds. *Recent Advances in Intrusion Detection (RAID 2005)*. Lecture Notes in Computer Science, vol. 3858. Berlin: Springer, pp.207-226.
- Kuhn, H.W., 1955. The Hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1-2), pp.83-97.
- Narayanan, A., Meng, G., Yang, L., Liu, J. and Chen, L., 2016. Contextual Weisfeiler-Lehman graph kernel for malware detection. In: *2016 International Joint Conference on Neural Networks (IJCNN)*. IEEE, pp.4701-4708.
- Neuhaus, M., Riesen, K. and Bunke, H., 2006. Fast suboptimal algorithms for the computation of graph edit distance. In: *Structural, Syntactic, and Statistical Pattern Recognition (SSPR/SPR 2006)*. Lecture Notes in Computer Science, vol. 4109. Berlin: Springer, pp.163-172.
- Riesen, K. and Bunke, H., 2009. Approximate graph edit distance computation by means of bipartite graph matching. *Image and Vision Computing*, 27(7), pp.950-959.
- Ugarte-Pedrero, X., Santos, I., García-Ferreira, I., Huerta, S., Sanz, B. and Bringas, P.G., 2014. On the adoption of anomaly detection for packed executable filtering. *Computers & Security*, 43, pp.126-144.
- Yakdan, K., Eschweiler, S., Gerhards-Padilla, E. and Smith, M., 2015. No more gotos: decompilation using pattern-independent control-flow structuring and semantic-preserving transformations. In: *Proceedings of the Network and Distributed System Security Symposium (NDSS 2015)*.
- Yan, J., Yan, G. and Jin, D., 2019. Classifying malware represented as control flow graphs using deep graph convolutional neural network. In: *2019 49th*

Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN). IEEE, pp.52-63.

Appendix A – Worked Example: GED computation for s01 vs s06

This section works through an example of GED computation under the uniform cost model for pair s01_clean and s06_medium_unroll. After canonical normalisation, s01_clean has a CFG with 4 nodes and role sequence [ENTRY, BRANCH, MERGE, EXIT], and s06_medium_unroll has a reduced CFG after chain collapsing with 5 BRANCH nodes, 4 EXIT nodes, and 1 ENTRY node (thirteen blocks before collapsing, then reduced after removing LINEAR chains). The initial state is s0 with g(s) = 0, mapping = {}, unmatched_g1 = {0, 1, 2, 3}, unmatched_g2 = {0..12}. The Hungarian algorithm heuristic computes the minimum cost of assigning each of these four g1 nodes to every g2 node and every deletion node. Under uniform costs, we can assign each of these nodes to its counterpart with zero cost, except for MERGE which has no counterpart in s06, which has no MERGE role nodes after unrolling. Thus, we assign it to BRANCH with unit cost 1. Thus, h(s_0) = 1, and therefore f(s_0) = 1.

The algorithm then expands states by assigning node 0 (ENTRY) of s01 to each of the unmatched nodes in s06, or by deleting it. The lowest-cost child matches node 1 (ENTRY) in s01 with the ENTRY node in s06 at cost 0. The cost of matching the remaining nodes in s01 with the unmatched nodes in s06 is given by the Hungarian bound h, which is 1. Therefore, f is 1. The algorithm then expands further by matching node 1 (BRANCH) in s01 with a BRANCH node in s06 at cost 0. Node 2 (MERGE) in s01 is then matched with either a BRANCH node in s06 at cost 1 or deleted at cost 1. Node 3 (EXIT) in s01 is then matched with an EXIT node in s06 at cost 0. The remaining nodes in s06, which are nine BRANCH nodes and three EXIT nodes, all have to be inserted at cost 1 each. Therefore, the cost is 12. The total cost is then 0 + 0 + 1 + 0 + 12 = 13. However, there is also the cost associated with edges. The GED is then 26.0.

This example demonstrates how the search works through a structured space of partial mappings, where the Hungarian heuristic helps to guide the search to regions of low cost. It also demonstrates why the GED increases with size difference, as the twelve unmatched s06 nodes all have a cost of 1, contributing to the total GED of 26.0.

Appendix B – Gen AI Declaration

ECS635U – Project Module
Generative AI Tools – Student Accountability Statement

The completed and signed statement must be included in the appendix of the progress and final reports.

Student Name: Artur Istratov
Project Title: Control Flow Graph Analysis using Graph Edit Distance for Cybersecurity Applications
Supervisor Name: Michael Shekelyan

1. Have you used Generative AI tools for your project?
YES
2. Please provide details of all the AI tools that you used in your project work.

AI Tool	Version	Publisher	Access URL
ChatGPT	5.5	OpenAI	https://chat.openai.com/

3. If you have used Artificial Intelligence (AI) tools in any part of your project please include in table format how, where, and why these tools were used. This should include references to the relevant parts of your project work — for example:

- Specific chapters or sections of the report (e.g. literature review, methodology, requirements)
- Software development (e.g. AI-assisted code generation, debugging, documentation)
- Interview or survey materials (e.g. AI-generated or refined questions/prompts)
- Datasets (e.g. AI-assisted data cleaning, labelling, or synthesis)
- The showcase video (e.g. AI-assisted scripting, narration, or editing support)

Be specific and transparent about your use of AI tools, including both direct outputs (e.g. text, code, images) and indirect uses (e.g. idea generation, editing assistance). The goal is to ensure academic integrity and provide a clear record of how AI contributed to your work.

Project Section	Description	Justification
Report planning and structure	Used Generative AI to help structure and organise my own ideas, including the logical order of sections and clarity of presentation.	I used GenAI as a supplementary planning tool to improve organisation and readability in terms of section organisation, while ensuring that the research, analysis, coding, and final content remained my own work.

3. Generative AI Student Declaration

- I have declared all uses of AI tools (e.g., coding assistants, text generators, data analysis tools) in this project.
- I have reviewed and verified all AI-assisted content to ensure accuracy and suitability.
- I take full responsibility for the originality and integrity of my project submission.
- I have referenced all sources, including those suggested or produced with AI support.
- I have not used AI in ways that undermine academic honesty (e.g., plagiarism, unverified code submission).
- My use of AI was supplementary and did not replace my own independent learning and problem-solving.

Signed: _Artur_Istratov_____ Date: ____06/05/2026_____